

TERRAFORM FINAL TASK

1. Create VPC
2. Create Internet gateway
3. Create Custom Route Table
4. Create Subnet
5. Associate subnet with Route Table
6. Create Security Group to allow port 22,80,443
7. Create a network interface with an ip in the subnet that was created in step 4
8. Assign an elastic IP to the network interface created in step 7
9. Create Ubuntu server and install/enable apache2

Complete Terraform step-by-step project that creates exactly what you listed, including a custom network interface and Elastic IP, then launches an Ubuntu server and installs Apache2.

Architecture

Internet

|

v

Elastic IP

|

v

Network Interface (Private IP)

|

v

Ubuntu EC2

|

v

Subnet

|

v

Custom Route Table ---> Internet Gateway

|

v

VPC

We will create:

1. VPC
2. Internet Gateway
3. Custom Route Table
4. Subnet
5. Route Table Association
6. Security Group — ports 22, 80, 443
7. Network Interface with private IP
8. Elastic IP attached to the Network Interface
9. Ubuntu EC2 + Apache2

Step 1: Create project directory

Open Git Bash / VS Code terminal:

```
mkdir terraform-ubuntu-apache
```

```
cd terraform-ubuntu-apache
```

Create these files:

```
terraform-ubuntu-apache/
```

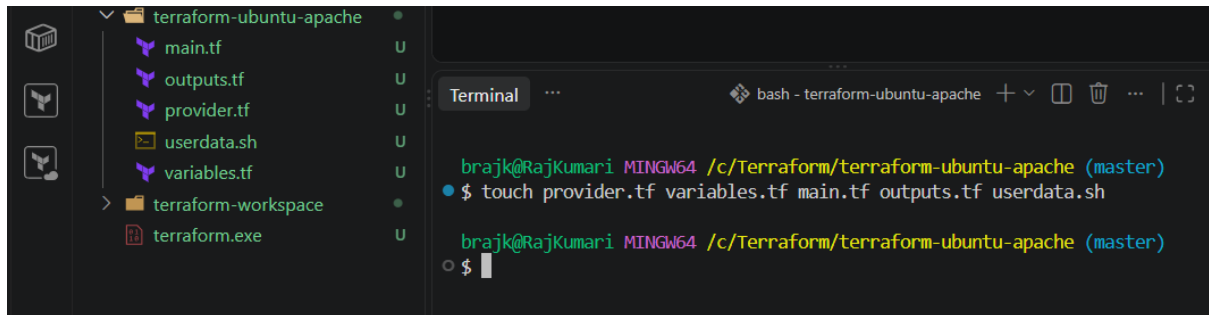
```
├── provider.tf
```

```
├── variables.tf
```

└── main.tf

└── outputs.tf

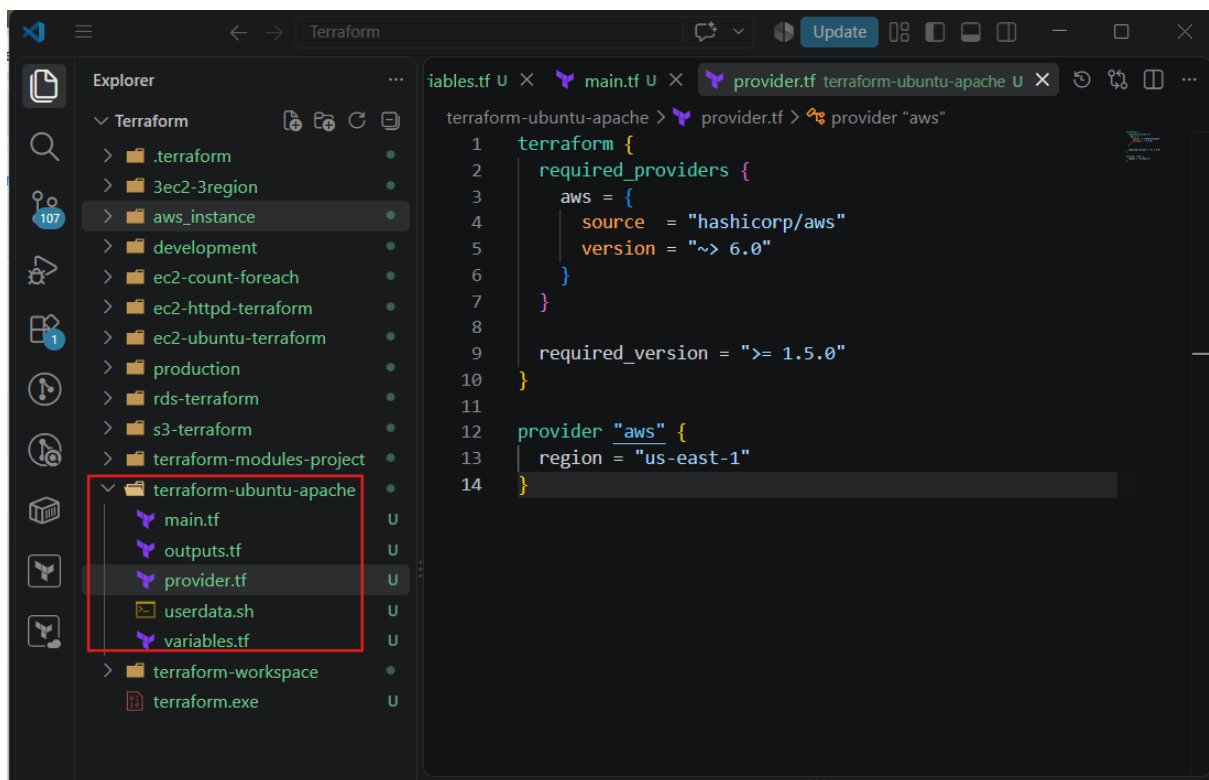
└── userdata.sh



Step 2: Create provider.tf

Create:

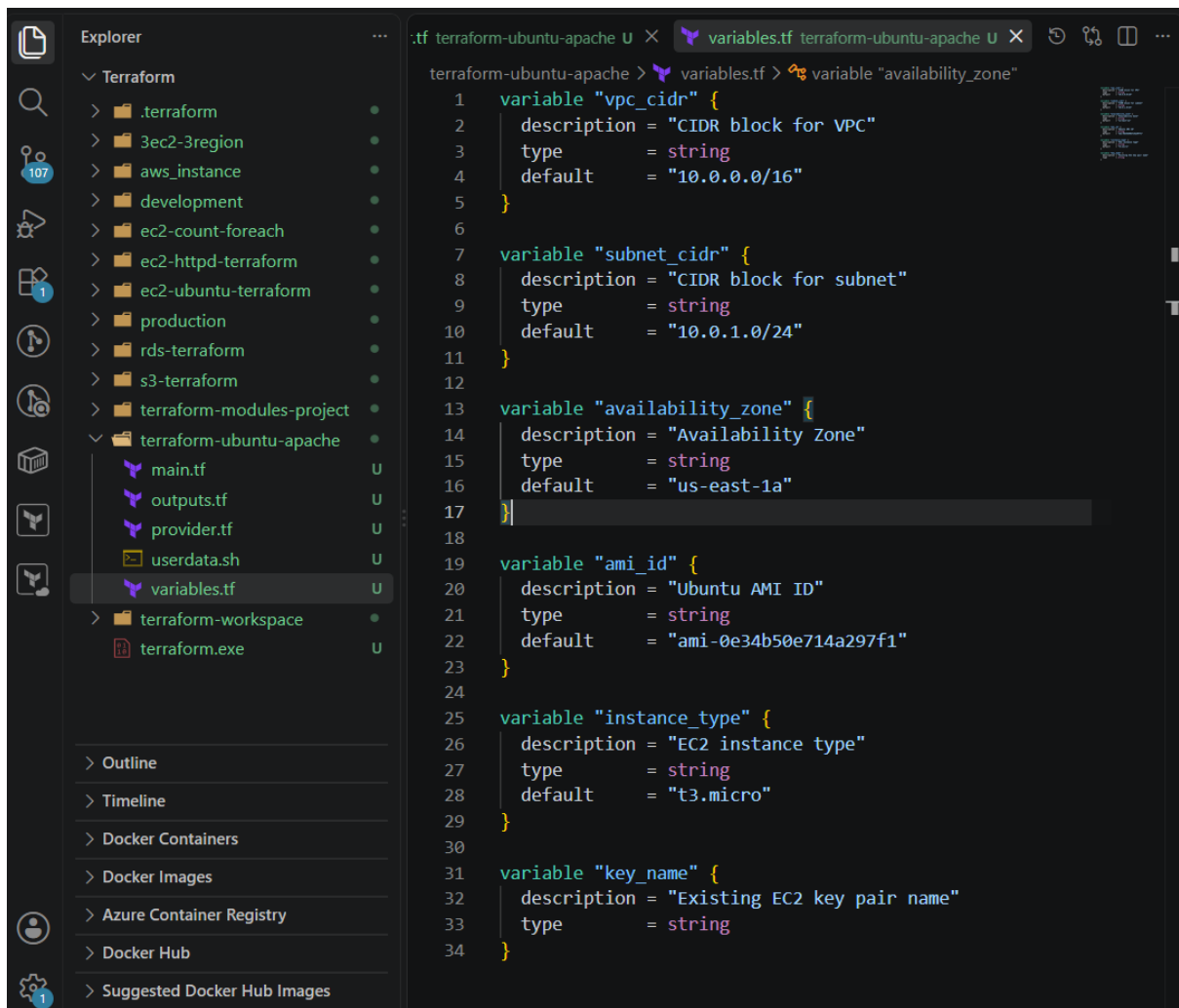
provider.tf



Step 3: Create variables

Create:

variables.tf



Important

The Ubuntu AMI ID is **region-specific**. The supplied AMI must exist in us-east-1.

If AWS says:

InvalidAMIID.NotFound

then you need to use a valid Ubuntu AMI for your AWS region.

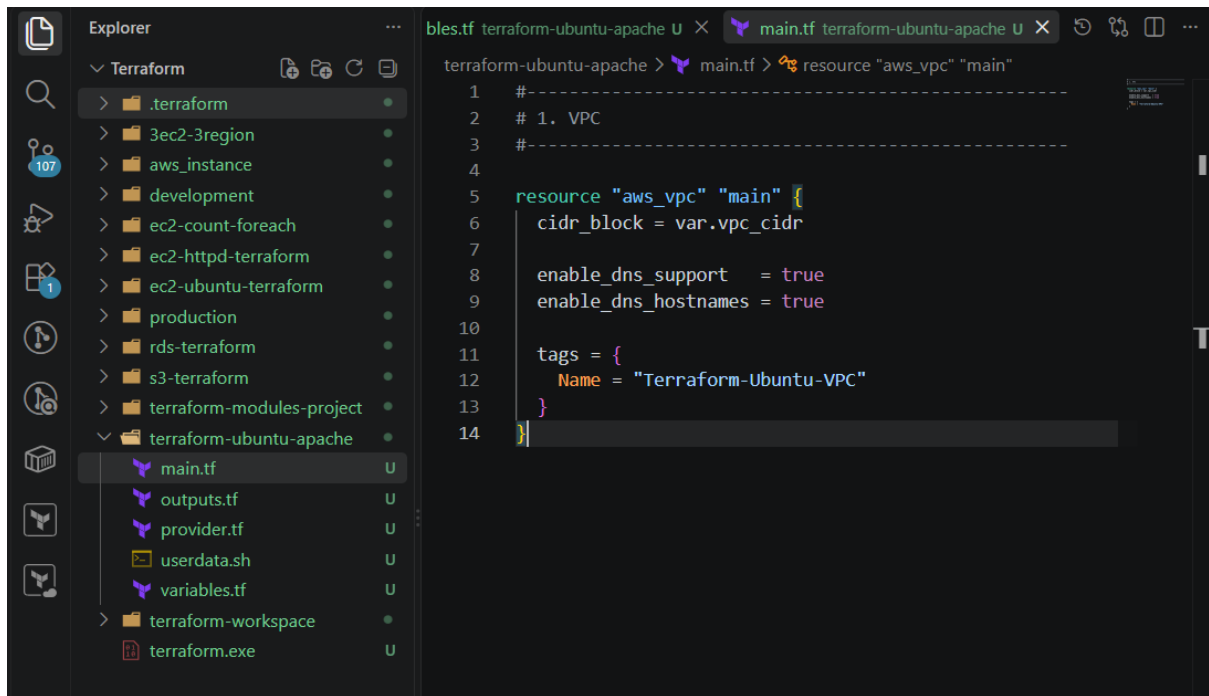
Step 4: Create VPC

Open:

main.tf

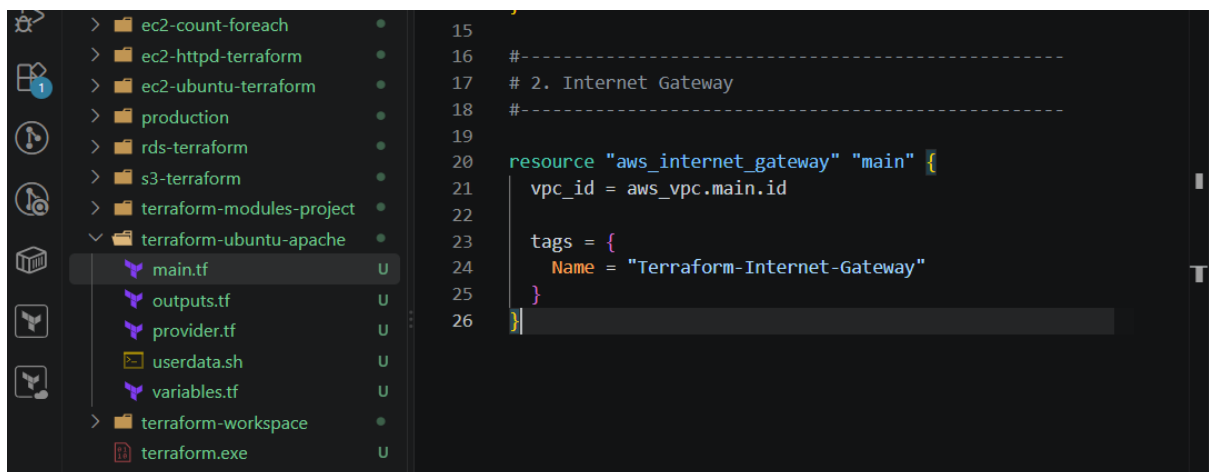
This creates:

VPC CIDR = 10.0.0.0/16



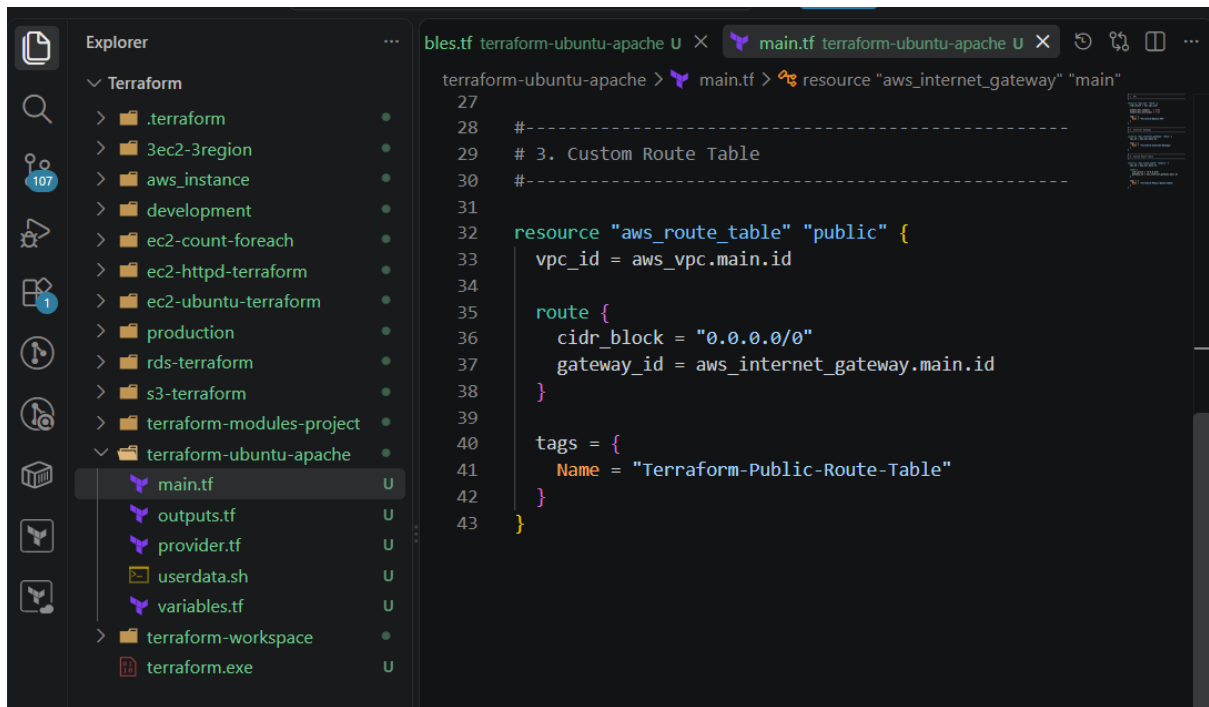
Step 5: Create Internet Gateway

Add to main.tf:



Step 6: Create Custom Route Table

Add:



The important part is:

```
cidr_block = "0.0.0.0/0"
```

It means:

All internet traffic

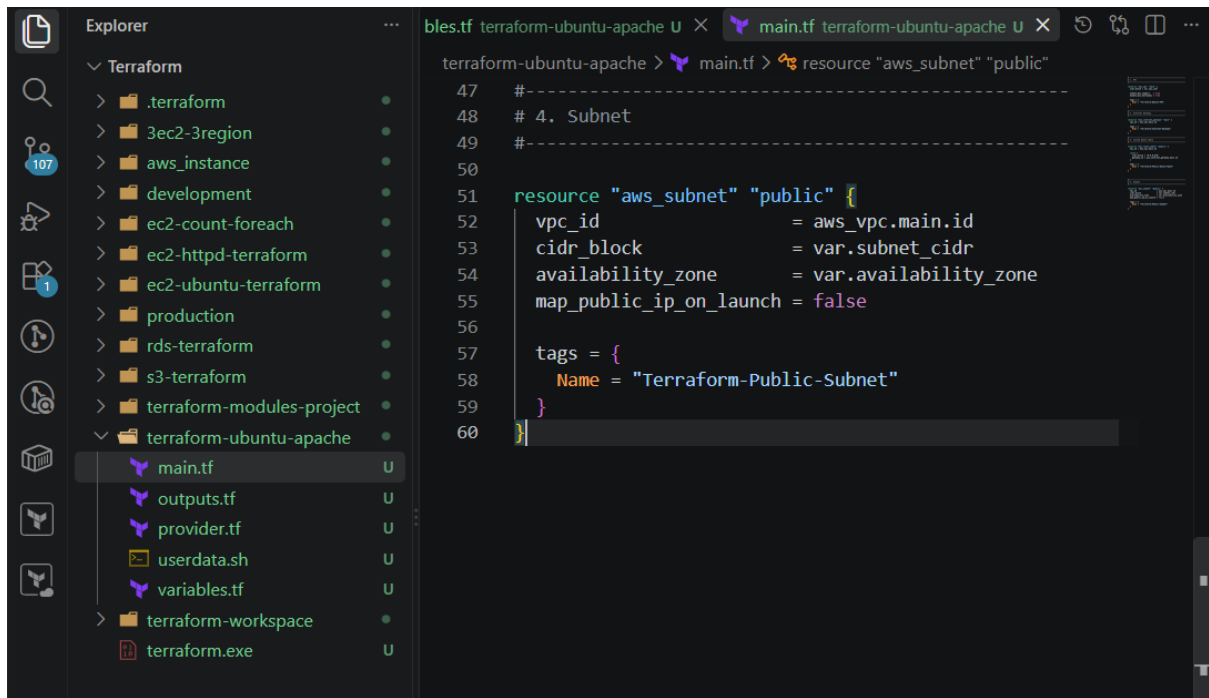
|

v

Internet Gateway

Step 7: Create Subnet

Add:



Our subnet is:

10.0.1.0/24

We intentionally use:

map_public_ip_on_launch = false

because we're going to attach an Elastic IP to the **network interface manually**, as requested.

Step 8: Associate Subnet with Route Table

Add:

Now:

Subnet

|

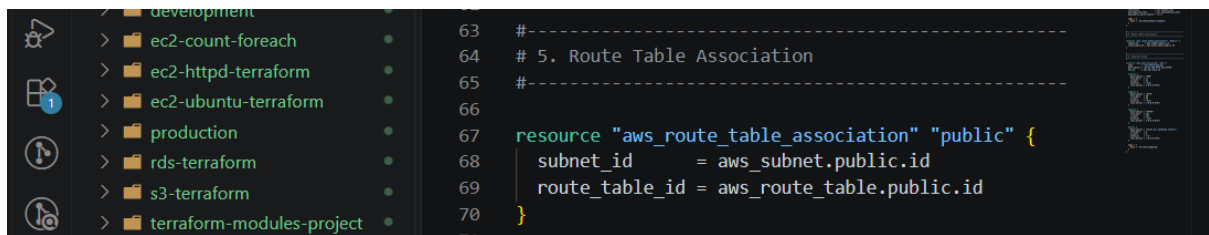
v

Route Table

|

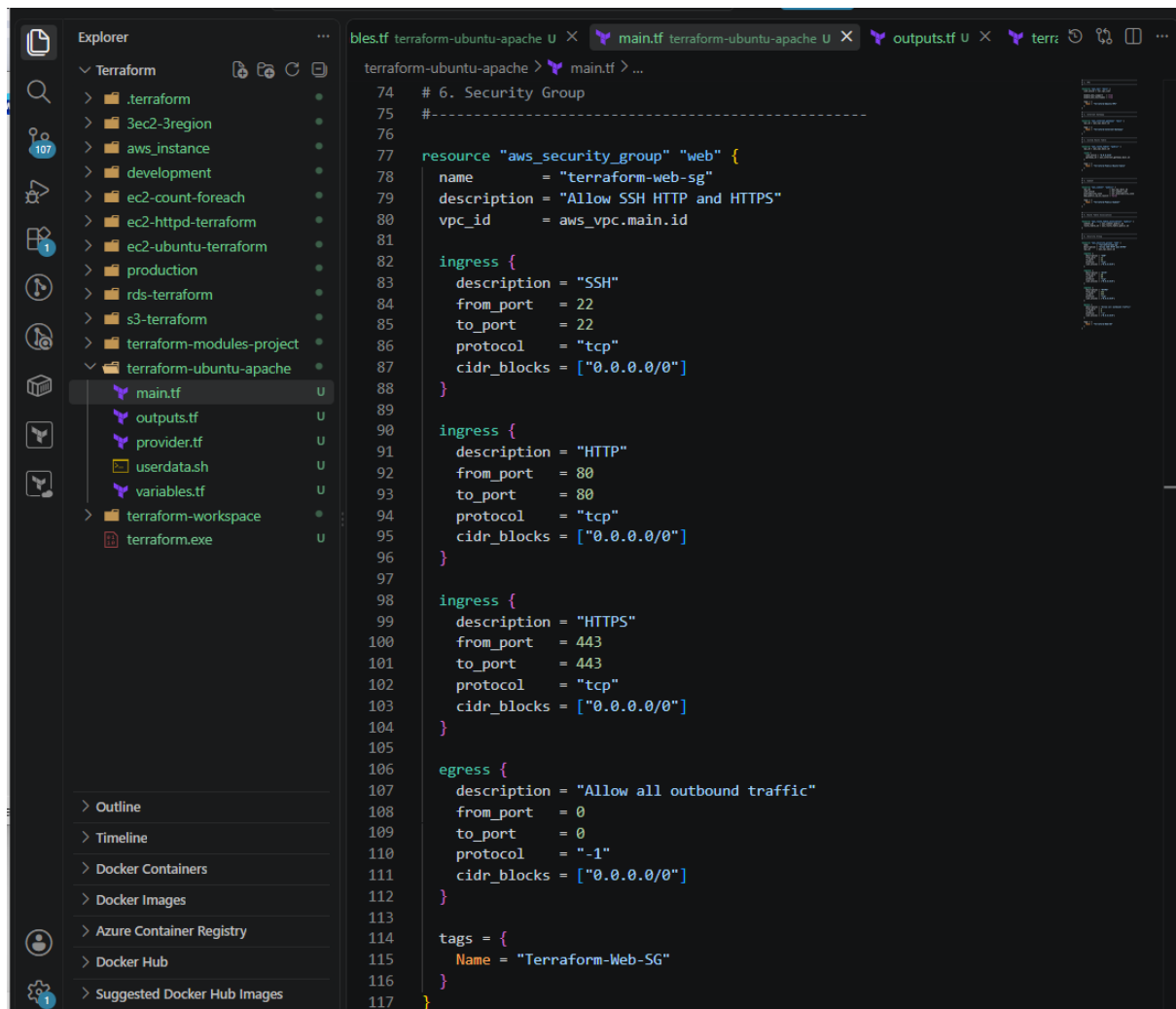
v

Internet Gateway



Step 9: Create Security Group

Add:



This allows:

Port Purpose

22 SSH

80 HTTP

443 HTTPS

Security note

For a real production server, don't normally expose SSH:

22 -> 0.0.0.0/0

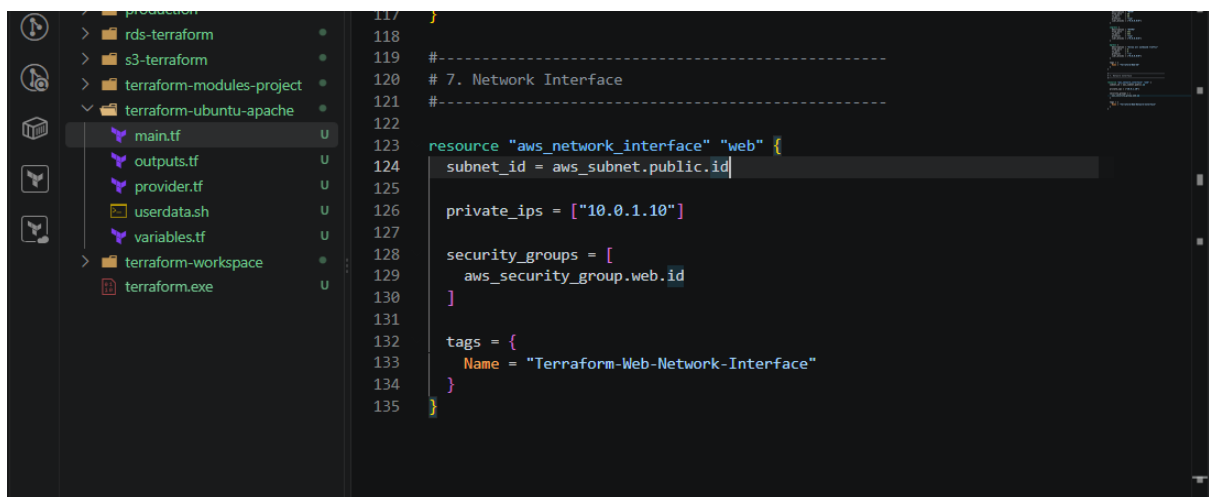
Instead, restrict it to your office/home public IP.

For your **lab**, we're keeping it open so you can connect easily.

Step 10: Create Network Interface

Now we create the network interface in the subnet.

Add:

A screenshot of a code editor with a dark theme. On the left, a file explorer shows a directory structure with folders like 'production', 'rds-terraform', 's3-terraform', 'terraform-modules-project', and 'terraform-ubuntu-apache'. Under 'terraform-ubuntu-apache', there are files: 'main.tf', 'outputs.tf', 'provider.tf', 'userdata.sh', and 'variables.tf'. The 'main.tf' file is selected. The main editor area shows Terraform code for an AWS network interface. The code is as follows:

```
117 }
118
119 #-----
120 # 7. Network Interface
121 #-----
122
123 resource "aws_network_interface" "web" {
124   subnet_id = aws_subnet.public.id
125
126   private_ips = ["10.0.1.10"]
127
128   security_groups = [
129     aws_security_group.web.id
130   ]
131
132   tags = {
133     Name = "Terraform-Web-Network-Interface"
134   }
135 }
```

Our network interface gets:

Private IP = 10.0.1.10

Architecture:

VPC

|

+-- Subnet 10.0.1.0/24

|

+-- ENI

|

+-- 10.0.1.10

Step 11: Create Elastic IP

Add:

This attaches the Elastic IP directly to our network interface.

So:

Elastic IP

|

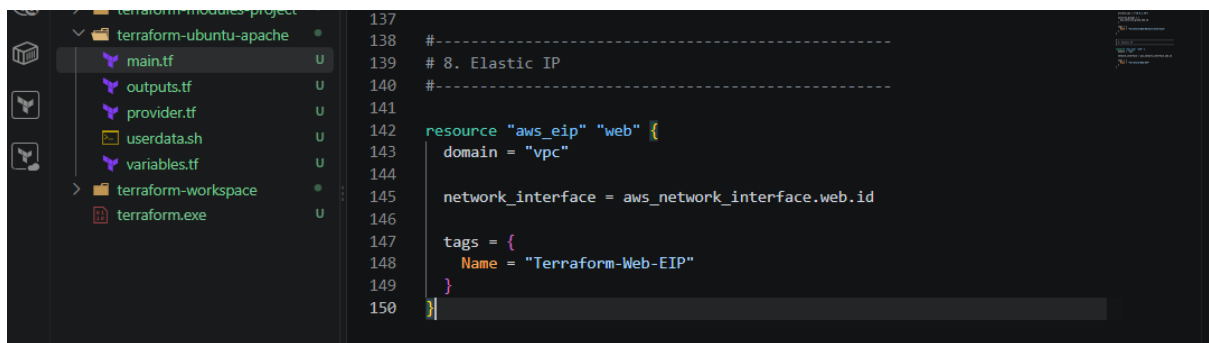
v

Network Interface

|

v

10.0.1.10



Step 12: Create User Data script

Now create:

userdata.sh

The screenshot shows a VS Code editor with a Terraform project. The Explorer sidebar on the left shows a directory tree under 'Terraform' with subdirectories like '.terraform', '3ec2-3region', 'aws_instance', 'development', 'ec2-count-foreach', 'ec2-httpd-terraform', 'ec2-ubuntu-terraform', 'production', 'rds-terraform', 's3-terraform', 'terraform-modules-project', and 'terraform-ubuntu-apache'. The 'terraform-ubuntu-apache' directory is expanded, showing files: 'main.tf', 'outputs.tf', 'provider.tf', 'variables.tf', and 'userdata.sh'. The 'main.tf' file is selected, and its content is displayed in the editor. The script is a shell script for installing Apache on Ubuntu. It includes comments for each step: updating packages, installing Apache, enabling Apache at boot, and starting Apache. It also includes a comment for creating a custom web page and the corresponding HTML code for 'index.html'.

```
terraform-ubuntu-apache > cat userdata.sh
1 #!/bin/bash
2
3 # Update packages
4 apt-get update -y
5
6 # Install Apache
7 apt-get install -y apache2
8
9 # Enable Apache at boot
10 systemctl enable apache2
11
12 # Start Apache
13 systemctl start apache2
14
15 # Create a custom web page
16 cat > /var/www/html/index.html <<EOF
17 <!DOCTYPE html>
18 <html>
19 <head>
20 |   <title>Terraform Apache Server</title>
21 </head>
22 <body>
23 |   <h1>Apache2 is running!</h1>
24 |   <h2>Created using Terraform</h2>
25 |   <p>Ubuntu EC2 Web Server</p>
26 </body>
27 </html>
28 EOF
```

This script automatically:

apt update



Install Apache2



Enable Apache2



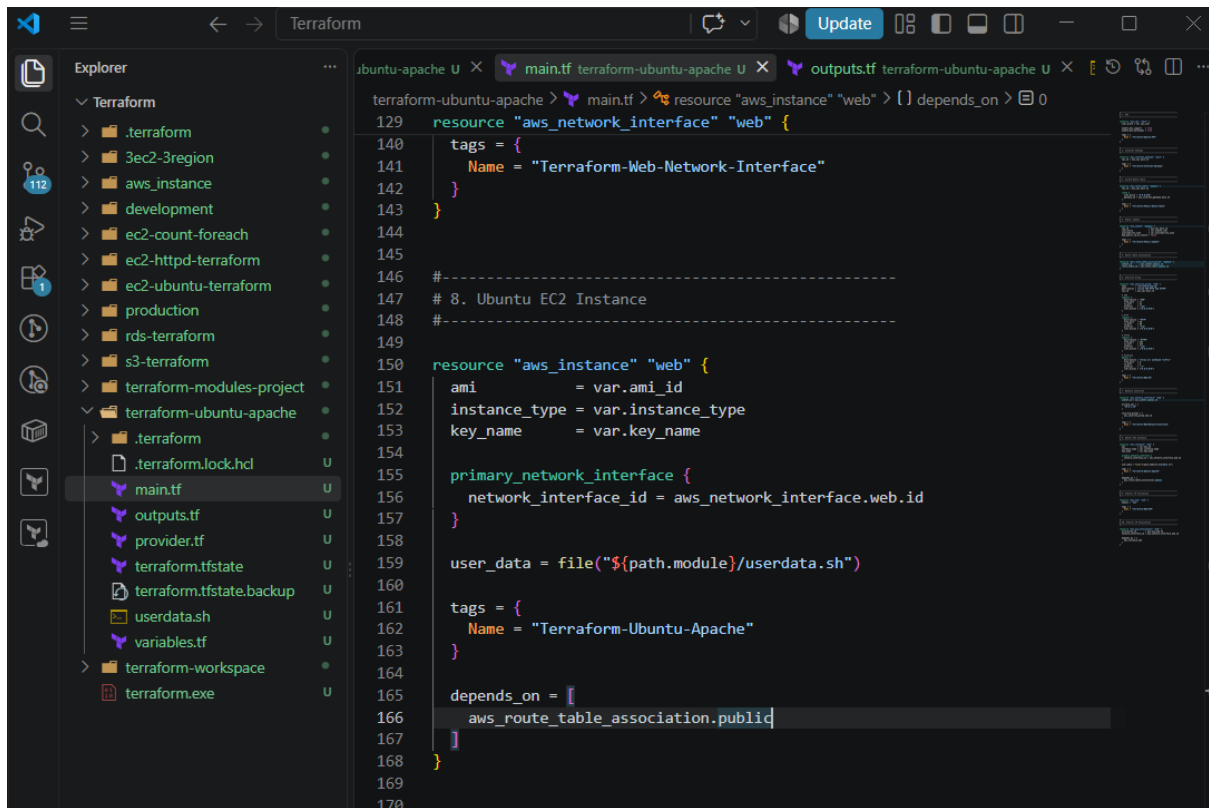
Start Apache2



Create index.html

Step 13: Create Ubuntu EC2 instance

Add to main.tf:



Important

Notice that we're **not** specifying:

subnet_id

or:

vpc_security_group_ids

inside aws_instance.

That's because we're attaching the EC2 instance to the **network interface we already created**.

The network interface already contains:

Subnet

Private IP

Security Group

Step 14: Complete main.tf

Your complete main.tf should now look like this:

```
#-----
```

1. VPC

#-----

```
resource "aws_vpc" "main" {
```

```
  cidr_block = var.vpc_cidr
```

```
  enable_dns_support = true
```

```
  enable_dns_hostnames = true
```

```
  tags = {
```

```
    Name = "Terraform-Ubuntu-VPC"
```

```
  }
```

```
}
```

#-----

2. Internet Gateway

#-----

```
resource "aws_internet_gateway" "main" {
```

```
  vpc_id = aws_vpc.main.id
```

```
  tags = {
```

```
    Name = "Terraform-Internet-Gateway"
```

```
  }
```

```
}
```

```
#-----
```

```
# 3. Custom Route Table
```

```
#-----
```

```
resource "aws_route_table" "public" {
```

```
  vpc_id = aws_vpc.main.id
```

```
  route {
```

```
    cidr_block = "0.0.0.0/0"
```

```
    gateway_id = aws_internet_gateway.main.id
```

```
  }
```

```
  tags = {
```

```
    Name = "Terraform-Public-Route-Table"
```

```
  }
```

```
}
```

```
#-----
```

```
# 4. Subnet
```

```
#-----
```

```
resource "aws_subnet" "public" {
```

```
  vpc_id      = aws_vpc.main.id
```

```
cidr_block      = var.subnet_cidr
availability_zone = var.availability_zone
map_public_ip_on_launch = false
```

```
tags = {
  Name = "Terraform-Public-Subnet"
}
}
```

```
#-----
```

```
# 5. Route Table Association
```

```
#-----
```

```
resource "aws_route_table_association" "public" {
  subnet_id    = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}
```

```
#-----
```

```
# 6. Security Group
```

```
#-----
```

```
resource "aws_security_group" "web" {
  name = "terraform-web-sg"
```

```
description = "Allow SSH HTTP and HTTPS"
```

```
vpc_id      = aws_vpc.main.id
```

```
ingress {
```

```
    description = "SSH"
```

```
    from_port   = 22
```

```
    to_port     = 22
```

```
    protocol    = "tcp"
```

```
    cidr_blocks = ["0.0.0.0/0"]
```

```
}
```

```
ingress {
```

```
    description = "HTTP"
```

```
    from_port   = 80
```

```
    to_port     = 80
```

```
    protocol    = "tcp"
```

```
    cidr_blocks = ["0.0.0.0/0"]
```

```
}
```

```
ingress {
```

```
    description = "HTTPS"
```

```
    from_port   = 443
```

```
    to_port     = 443
```

```
    protocol    = "tcp"
```

```
    cidr_blocks = ["0.0.0.0/0"]
```

```
}
```

```
egress {  
    description = "Allow all outbound traffic"  
    from_port   = 0  
    to_port     = 0  
    protocol    = "-1"  
    cidr_blocks = ["0.0.0.0/0"]  
}
```

```
tags = {  
    Name = "Terraform-Web-SG"  
}  
}
```

```
#-----
```

```
# 7. Network Interface
```

```
#-----
```

```
resource "aws_network_interface" "web" {  
    subnet_id = aws_subnet.public.id  
  
    private_ips = ["10.0.1.10"]  
  
    security_groups = [  
        aws_security_group.web.id  
    ]  
}
```

```
]
```

```
tags = {  
  Name = "Terraform-Web-Network-Interface"  
}  
}
```

```
#-----
```

```
# 8. Elastic IP
```

```
#-----
```

```
resource "aws_eip" "web" {  
  domain = "vpc"  
  
  network_interface = aws_network_interface.web.id  
  
  tags = {  
    Name = "Terraform-Web-EIP"  
  }  
}
```

```
#-----
```

```
# 9. Ubuntu EC2 Instance
```

```
#-----
```

```
resource "aws_instance" "web" {  
    ami          = var.ami_id  
  
    instance_type = var.instance_type  
  
    key_name     = var.key_name  
  
    network_interface {  
        network_interface_id = aws_network_interface.web.id  
  
        device_index      = 0  
    }  
  
    user_data = file("${path.module}/userdata.sh")  
  
    tags = {  
        Name = "Terraform-Ubuntu-Apache"  
    }  
}
```

Step 15: Create outputs

Create:

outputs.tf

Add:

```
output "vpc_id" {  
    value = aws_vpc.main.id  
}
```

```
output "subnet_id" {  
    value = aws_subnet.public.id  
}
```

```
output "route_table_id" {  
    value = aws_route_table.public.id  
}
```

```
output "security_group_id" {  
    value = aws_security_group.web.id  
}
```

```
output "network_interface_id" {  
    value = aws_network_interface.web.id  
}
```

```
output "private_ip" {  
    value = aws_network_interface.web.private_ip  
}
```

```
output "elastic_ip" {  
    value = aws_eip.web.public_ip  
}
```

```
output "instance_id" {  
    value = aws_instance.web.id  
}
```

```
}
```

```
output "apache_url" {  
  value = "http://${aws_eip.web.public_ip}"  
}
```

```
output "ssh_command" {  
  value = "ssh -i your-key.pem ubuntu@${aws_eip.web.public_ip}"  
}
```

Step 16: Check your files

You should now have:

terraform-ubuntu-apache/

|

├── provider.tf

├── variables.tf

├── main.tf

├── outputs.tf

└── userdata.sh

Step 17: Terraform initialization

Run:

```
terraform init
```

You should see something similar to:

Initializing the backend...

Initializing provider plugins...

- Installing hashicorp/aws...

Terraform has been successfully initialized!

Step 18: Format Terraform

Run:

```
terraform fmt
```

Expected:

```
provider.tf
```

```
variables.tf
```

```
main.tf
```

```
outputs.tf
```

Step 19: Validate

Run:

```
terraform validate
```

Expected:

Success! The configuration is valid.

Step 20: Create your EC2 key pair

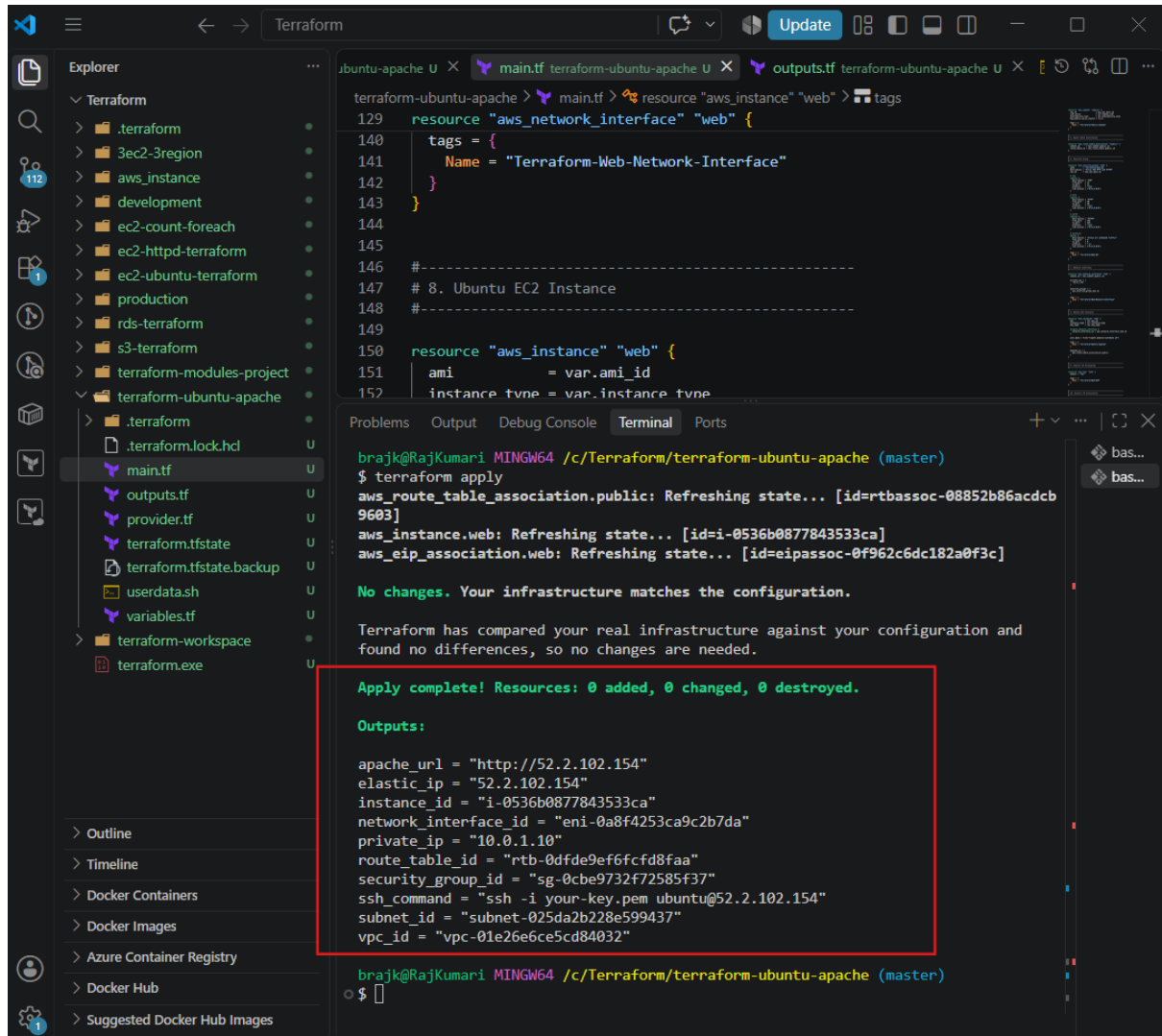
Before applying, make sure you already have an EC2 key pair in **AWS → EC2 → Key Pairs**.

For example:

terraform-key

Then provide it to Terraform.

You have two options.



The screenshot shows a VS Code editor with a Terraform project. The Explorer pane on the left shows the project structure, including a directory named 'terraform-ubuntu-apache' which contains files like '.terraform', 'main.tf', 'outputs.tf', 'provider.tf', 'terraform.tfstate', 'terraform.tfstate.backup', 'userdata.sh', 'variables.tf', 'terraform-workspace', and 'terraform.exe'. The main editor shows the 'main.tf' file with Terraform code for creating an AWS network interface and an EC2 instance. The terminal at the bottom shows the output of the 'terraform apply' command, indicating that the infrastructure matches the configuration and no changes are needed. A red box highlights the 'Outputs' section of the terminal output.

```
terraform-ubuntu-apache > main.tf > resource "aws_instance" "web" > tags
129 resource "aws_network_interface" "web" {
140     tags = {
141         Name = "Terraform-Web-Network-Interface"
142     }
143 }
144
145
146 #-----
147 # 8. Ubuntu EC2 Instance
148 #-----
149
150 resource "aws_instance" "web" {
151     ami           = var.ami_id
152     instance_type = var.instance_type
153 }
```

```
brajk@RajKumari MINGW64 /c/Terraform/terraform-ubuntu-apache (master)
$ terraform apply
aws_route_table_association.public: Refreshing state... [id=rtbassoc-08852b86acdcb9603]
aws_instance.web: Refreshing state... [id=i-0536b0877843533ca]
aws_eip_association.web: Refreshing state... [id=eipassoc-0f962c6dc182a0f3c]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and
found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

apache_url = "http://52.2.102.154"
elastic_ip = "52.2.102.154"
instance_id = "i-0536b0877843533ca"
network_interface_id = "eni-0a8f4253ca9c2b7da"
private_ip = "10.0.1.10"
route_table_id = "rtb-0dfde9ef6fcfd8faa"
security_group_id = "sg-0cbe9732f72585f37"
ssh_command = "ssh -i your-key.pem ubuntu@52.2.102.154"
subnet_id = "subnet-025da2b228e599437"
vpc_id = "vpc-01e26e6ce5cd84032"

brajk@RajKumari MINGW64 /c/Terraform/terraform-ubuntu-apache (master)
o $
```

Step 21: Terraform Plan

Run:

terraform plan

You should see resources similar to:

Plan: 9 to add, 0 to change, 0 to destroy.

The resources are:

1. VPC

2. Internet Gateway
 3. Route Table
 4. Subnet
 5. Route Table Association
 6. Security Group
 7. Network Interface
 8. Elastic IP
 9. EC2 Instance
-

Step 22: Apply

Run:

```
terraform apply
```

Terraform asks:

Do you want to perform these actions?

Only 'yes' will be accepted to approve.

Enter a value:

Enter:

yes

Terraform will create the infrastructure.

Step 23: Check Terraform outputs

After successful creation:

```
terraform output
```

You should get something like:

```
elastic_ip = "54.xx.xx.xx"
```

`instance_id = "i-xxxxxxxxxxxxxxxxxx"`

`network_interface_id = "eni-xxxxxxxxxxxxxxxxxx"`

`private_ip = "10.0.1.10"`

`subnet_id = "subnet-xxxxxxxxxxxxxxxxxx"`

`vpc_id = "vpc-xxxxxxxxxxxxxxxxxx"`

Most importantly:

`elastic_ip = "54.xx.xx.xx"`

Step 24: Test Apache

Open your browser:

`http://YOUR_ELASTIC_IP`

For example:

`http://54.xx.xx.xx`

You should see:

Apache2 is running!

Created using Terraform

Ubuntu EC2 Web Server

Step 25: SSH into Ubuntu

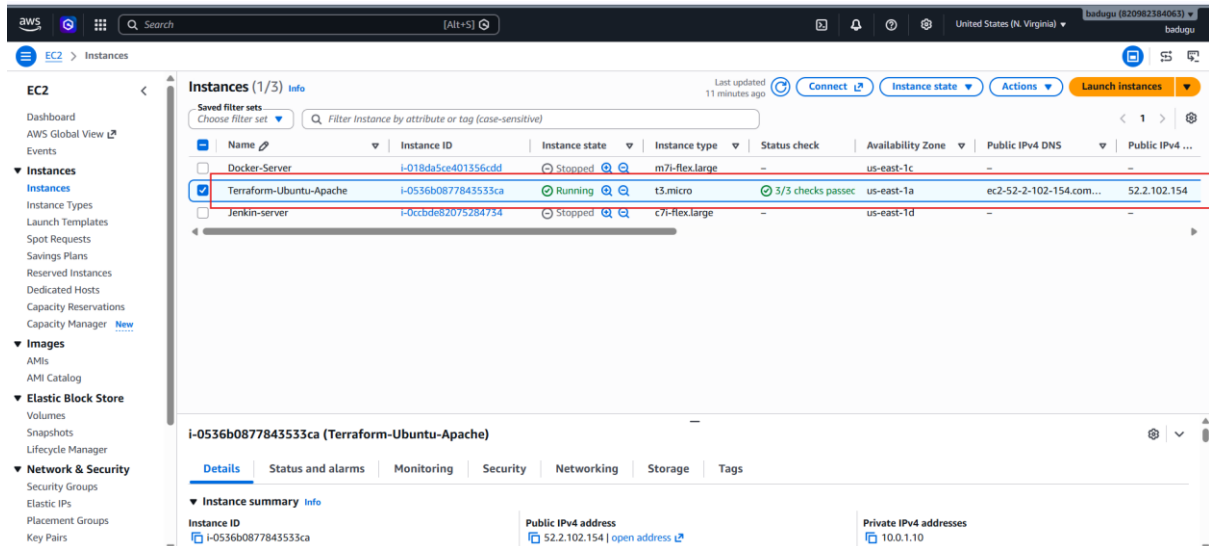
Use the Elastic IP:

```
ssh -i terraform-key.pem ubuntu@YOUR_ELASTIC_IP
```

Example:

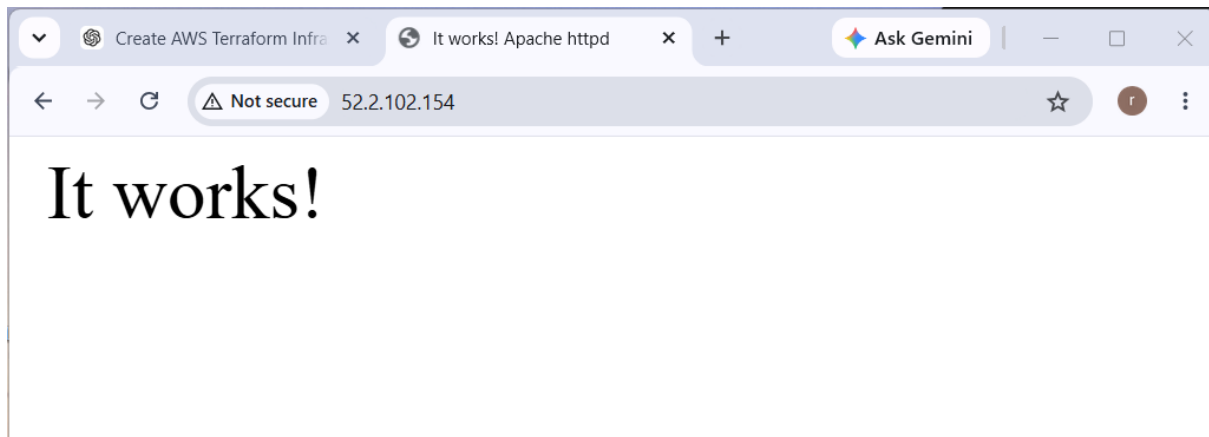
```
ssh -i terraform-key.pem ubuntu@54.xx.xx.xx
```

If your .pem is in another directory, specify its full path.

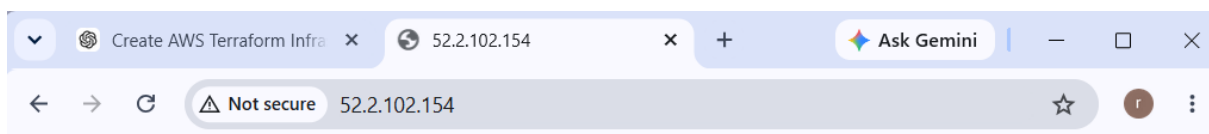


```
Complete!
[ec2-user@ip-10-0-1-10 ~]$ sudo systemctl start httpd
[ec2-user@ip-10-0-1-10 ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[ec2-user@ip-10-0-1-10 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-09-17 07:26:05 UTC; 36s ago
     Docs: man:httpd.service(8)
  Main PID: 28600 (httpd)
    Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served: 0"
    Tasks: 177 (limit: 1059)
   Memory: 13.8M
      CPU: 104ms
    CGroup: /system.slice/httpd.service
            └─28600 /usr/sbin/httpd -DFOREGROUND
              └─28619 /usr/sbin/httpd -DFOREGROUND
                └─28623 /usr/sbin/httpd -DFOREGROUND
                  └─28642 /usr/sbin/httpd -DFOREGROUND
                    └─28646 /usr/sbin/httpd -DFOREGROUND

Sep 17 07:26:05 ip-10-0-1-10.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server ->
Sep 17 07:26:05 ip-10-0-1-10.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server ->
Sep 17 07:26:05 ip-10-0-1-10.ec2.internal httpd[28600]: Server configured, listening on: 0.0.0.0:80
[ec2-user@ip-10-0-1-10 ~]$
```



```
ec2-user@ip-10-0-1-10:~$ echo "<h1>Apache is running on Amazon Linux 2023.....Rajkumari</h1>" | sudo tee /var/www/html/index.html
[ec2-user@ip-10-0-1-10 ~]$
```



Apache is running on Amazon Linux 2023.....Rajkumari

